



RIDDY

Resumo executivo e plano de produção

Arquitetura, tecnologias, credenciais, conexões, hospedagem e roteiro de go-live

STATUS	18/18 etapas concluídas
--------	-------------------------

Documento preparado em 28 de agosto de 2026
Versão 1.0 - uso interno e planejamento de implantação

A Riddy é um marketplace P2P de aluguel de carros e motocicletas. O MVP cobre a jornada pública, cadastro e autenticação, reservas, checkout, pagamento, KYC, área do anfitrião, mensagens e administração.

Dimensão	Situação	Leitura prática
Produto	Concluído	As 18 etapas funcionais e de hardening foram implementadas.
Código	Aprovado	Build, lint, tipos, migrações e suítes automatizadas foram aprovados.
Homologação	Pronto	Pode receber usuários e credenciais de teste em ambiente controlado.
Produção	Condicional	Depende de infraestrutura, segredos, contratos, testes reais e operação.

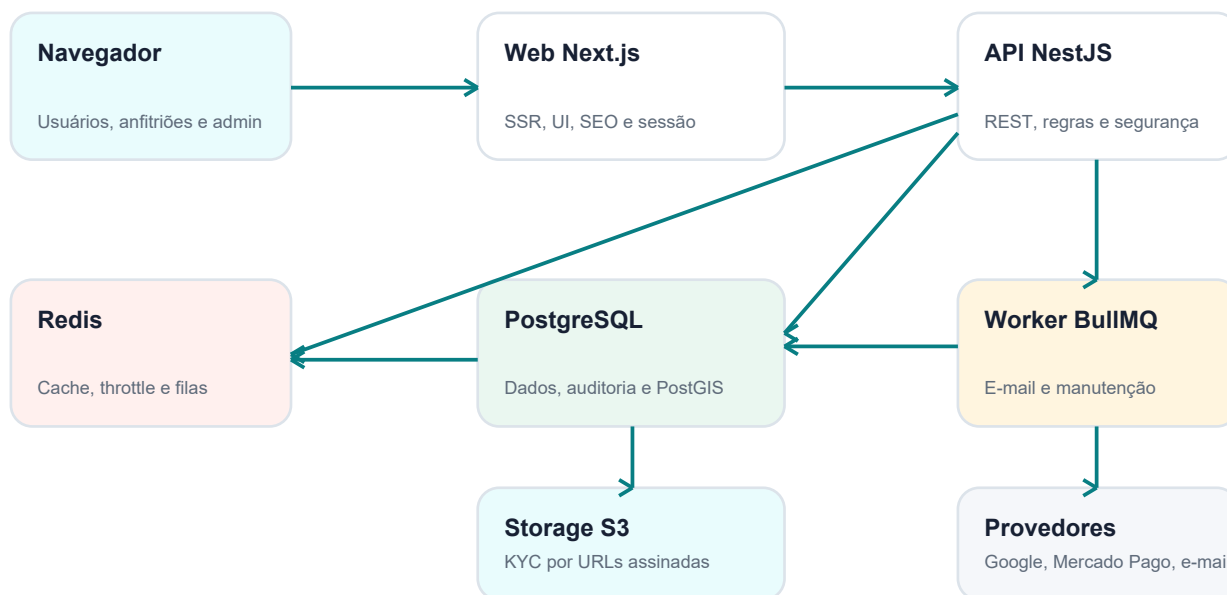
Recomendação principal

Implantar a primeira produção em Render: Web Next.js, API NestJS, worker BullMQ, PostgreSQL/PostGIS e Key Value Redis. Para KYC, usar AWS S3 privado com versionamento, criptografia e política de retenção. Configurar deploy somente após a CI do GitHub passar.

O Supabase é tecnicamente compatível com PostgreSQL/PostGIS e oferece storage S3, mas não substitui sozinho a API permanente, o Redis e o worker BullMQ. Ele pode ser usado como componente, não como hospedagem completa da arquitetura atual.

ESCOPO DO MVP

Público e locatário	Anfitrião	Operação
Home, catálogo, detalhes, conta, Google OAuth, perfil, reserva, checkout, cartão/Pix, KYC e mensagens.	Onboarding, veículos, disponibilidade, reservas recebidas, calendário, financeiro e mensagens.	Admin, analista KYC, auditoria, Redis, filas BullMQ, health checks, hardening e CI.



O navegador conversa com o Web Next.js e com a API REST. O PostgreSQL é a fonte de verdade; Redis atende cache, throttling e BullMQ; arquivos KYC vão diretamente ao storage por URLs assinadas. O worker processa recuperação de senha e limpeza de autenticação.

PRINCÍPIOS JÁ APLICADOS

- Sessão por cookie HttpOnly, token opaco e revogação no banco.
- API separada do frontend, DTOs validados e autorização por papel.
- Idempotência em pagamento e validação de assinatura nos webhooks.
- Bucket KYC privado; leitura e escrita somente por URLs temporárias.
- Redis compartilhado para múltiplas instâncias e worker separável da API.
- Liveness sem dependências e readiness validando PostgreSQL e Redis.

Limite arquitetural atual

A aplicação ainda não tem mapas interativos, antivírus de uploads, provedor de e-mail definitivo, observabilidade externa, teste de carga ou política jurídica final para KYC. Esses pontos não impedem homologação, mas bloqueiam go-live irrestrito.

Tecnologias utilizadas

Stack verificada diretamente no monorepo

Camada	Tecnologia	Uso na Riddy
Frontend	Next.js 16, React 19, TypeScript 5	SSR/App Router, páginas públicas e áreas autenticadas.
UI	Tailwind CSS 4, Radix UI, CVA, Lucide	Design system responsivo e componentes acessíveis.
Backend	NestJS 11 sobre Fastify 5	API REST, Swagger/OpenAPI, guards e regras de negócio.
Banco	PostgreSQL 17 + PostGIS 3.5	Dados transacionais, geoespacial, auditoria e idempotência.
ORM	Drizzle ORM + Drizzle Kit	Schema tipado, migrações e verificação de integridade.
Cache/filas	Redis 8 + ioredis + BullMQ	Cache, rate limiting, jobs e manutenção assíncrona.
Storage	S3 API + AWS SDK; MinIO local	KYC privado, presigned URLs, validação e checksums.
Integrações	Google OAuth 2.0 + Mercado Pago	Login social, cartão, Pix, webhooks e reembolso.
Qualidade	Jest, ESLint, Prettier, TypeScript strict	Testes, lint, formatação e checagem estática.
Monorepo	pnpm 11 + Turborepo	Builds e comandos coordenados para Web e API.
Infra local	Docker Compose	PostGIS, Redis e MinIO para desenvolvimento.
Entrega	GitHub Actions + Dependabot	CI com testes, auditoria, migrações e builds.

Versões de runtime

O package.json exige Node.js >= 22.13.0 e pnpm 11.23.0. A CI deve usar Node 22.x para permanecer dentro da linha LTS e compatível com o pnpm configurado.

PORTAS LOCAIS

Serviço	Porta	Uso
Web	3000	Interface Next.js
API	4000	REST, OAuth callback e Swagger local
PostgreSQL	5433	Banco local exposto pelo Docker
Redis	6379	Cache, throttle e BullMQ
MinIO	9000 / 9001	API S3 / console local

Credenciais e conexões pendentes

Levantamento seguro: somente presença e finalidade, sem copiar valores secretos

Bloco	Situação	O que providenciar
Domínio e TLS	Pendente	Domínio raiz, app e api; DNS; certificados; WEB_URL, CORS_ORIGIN e URLs públicas HTTPS.
Google OAuth	Local presente	Criar cliente/projeto produtivo separado, callback HTTPS, domínio verificado, consent screen e política de privacidade.
Mercado Pago	Ausente	Public Key, Access Token, Webhook Secret e URL de webhook HTTPS; primeiro TEST, depois produção.
E-mail/reset	Ausente	AUTH_RESET_WEBHOOK_URL e AUTH_RESET_WEBHOOK_SECRET apontando para um serviço real de entrega.
PostgreSQL	Somente local	DATABASE_URL produtiva com SSL, senha forte, PostGIS, pool, backup e restauração testada.
Redis	Somente padrão local	REDIS_URL com TLS/rede privada, autenticação, persistência e política noeviction.
Storage KYC	Somente padrão local	Endpoint/região/bucket/chaves S3 produtivas, bucket privado, CORS, criptografia e retenção.
Observabilidade	Ausente	Destino de logs, alertas, uptime, métricas e canal de incidentes.
Segredos CI/CD	Pendente	Variáveis no provedor; nunca em .env versionado ou em NEXT_PUBLIC_* quando forem privadas.

O que já está protegido

Os arquivos .env e .env.local estão ignorados pelo Git. O Google Client ID e Client Secret foram detectados como preenchidos localmente, mas seus valores não foram lidos para este relatório. O callback ainda aponta para localhost e não serve para produção.

VARIÁVEIS QUE DEVEM SER PÚBLICAS

NEXT_PUBLIC_API_URL, NEXT_PUBLIC_SITE_URL, NEXT_PUBLIC_STORAGE_ORIGIN, NEXT_PUBLIC_GOOGLE_AUTH_ENABLED e a Public Key do Mercado Pago quando fornecida por endpoint seguro. Tudo que começa com NEXT_PUBLIC_ é incorporado ao bundle do navegador.

VARIÁVEIS QUE DEVEM SER SECRETAS

GOOGLE_CLIENT_SECRET, MERCADO_PAGO_ACCESS_TOKEN, MERCADO_PAGO_WEBHOOK_SECRET, AUTH_RESET_WEBHOOK_SECRET, DATABASE_URL, REDIS_URL, S3_ACCESS_KEY e S3_SECRET_KEY. Armazenar somente no cofre de segredos do provedor.

Mapa de conexões produtivas

URLs, direção do tráfego e controles obrigatórios

Fluxo	Destino	Controle
Usuário -> Web	https://app.riddy.com.br	TLS, CSP, cookies Secure, noindex em áreas privadas.
Web -> API	https://api.riddy.com.br/api/v1	CORS exato, credentials=true, HTTPS e timeout.
Google -> API	/auth/google/callback	Redirect URI idêntica à cadastrada; state + PKCE.
Mercado Pago -> API	/payments/webhook/mercado-pago	HTTPS, x-signature, x-request-id, idempotência.
API/worker -> Postgres	Rede privada + SSL	Usuário exclusivo, pool limitado, migrações controladas.
API/worker -> Redis	Rede privada + TLS	Senha, noeviction, persistência e monitoramento de filas.
Navegador -> S3	URL assinada temporária	Bucket privado, CORS limitado ao Web, expiração curta.
API -> provedor de e-mail	Webhook HTTPS	Bearer secreto, retries BullMQ e logs sem token de reset.

ENDPOINTS OPERACIONAIS

Endpoint	Uso no provedor	Resultado esperado
GET /api/v1/health/live	Liveness	200 enquanto o processo estiver respondendo.
GET /api/v1/health/ready	Readiness/deploy	200 somente com PostgreSQL e Redis saudáveis.
GET /api/v1/health	Monitoramento	Estado, latência, versão e uptime.

Regra de exposição

Banco, Redis e worker não devem ter portas públicas. Somente Web, API e endpoints de webhook ficam acessíveis externamente. Swagger deve permanecer desabilitado em produção, salvo decisão operacional explícita.

Supabase é ideal?

Boa peça de infraestrutura, mas não é a plataforma completa para a arquitetura atual

Necessidade da Riddy	Supabase	Avaliação
PostgreSQL + PostGIS	Suporta a extensão PostGIS.	Bom encaixe para o banco, após revisar schemas e migrações.
Storage S3 privado	Compatível com S3 e presigned URLs.	Compatível, mas a documentação informa que não há versionamento S3.
API NestJS permanente	Não é o foco do produto.	Ainda precisa Render, Railway, Fly.io ou contêiner próprio.
Worker BullMQ contínuo	Edge Functions têm duração máxima.	Não substituem um consumidor BullMQ permanente.
Redis	Não é datastore nativo do projeto.	Exige Upstash, Render Key Value, Railway Redis ou equivalente.
Backups completos	Backup do banco não inclui objetos do Storage.	Exige política separada de backup para documentos KYC.
Auth	Possui Supabase Auth.	Seria redundante: a Riddy já tem autenticação própria e Google OAuth.

Veredito

Não usar Supabase como solução única. Ele é viável se a decisão for usar somente Supabase Postgres e/ou Storage, aceitando manter API, worker e Redis em outro provedor. Para reduzir migração e dispersão operacional, Render ou Railway encaixam melhor no MVP atual.

Quando escolher Supabase

- Quando o time valorizar muito o painel de banco, PostGIS e ferramentas de dados.
- Quando aceitar operar pelo menos dois provedores e backups separados para banco e objetos.
- Quando houver plano para manter a autenticação atual ou executar uma migração de auth explicitamente.

Quando não escolher

- Quando o objetivo for hospedar todo o monorepo, Redis e BullMQ em um único projeto.
- Quando versionamento/recuperação de objetos KYC for requisito imediato sem uma camada adicional.

Hospedagem recomendada

Comparação orientada à arquitetura, não somente ao plano gratuito

Opção	Indicação	Vantagens	Atenções
Render + AWS S3	Melhor para produção inicial	Web, API, worker, Postgres/PostGIS, Redis compatível, health checks, deploy após CI; S3 robusto para KYC.	Mais de um fornecedor e configuração inicial maior.
Railway completo	Melhor para rapidez	Monorepo, serviços, Postgres, Redis e bucket S3 compatível no mesmo projeto.	Requer disciplina explícita de backup/restauração e validação operacional.
Vercel + Render/AWS	Melhor frontend	Experiência excelente para Next.js, previews e CDN; backend/worker seguem fora.	Arquitetura distribuída e custo/segredos em mais painéis.
Supabase + compute externo	Bom para dados	PostGIS e storage integrados ao ecossistema Supabase.	Ainda exige API, worker e Redis externos; storage sem versionamento S3.
VPS com Docker	Maior controle	Pode executar tudo em uma VM ou cluster.	Maior risco operacional: patching, backup, TLS, HA e incidentes ficam com o time.

DECISÃO PARA A RIDDY

Escolha: Render + AWS S3

É o conjunto com menor mudança no código e melhor separação de responsabilidades: serviços Node permanentes, background worker compatível com BullMQ, Postgres com PostGIS, Redis compatível e storage com recursos fortes de proteção de documentos.

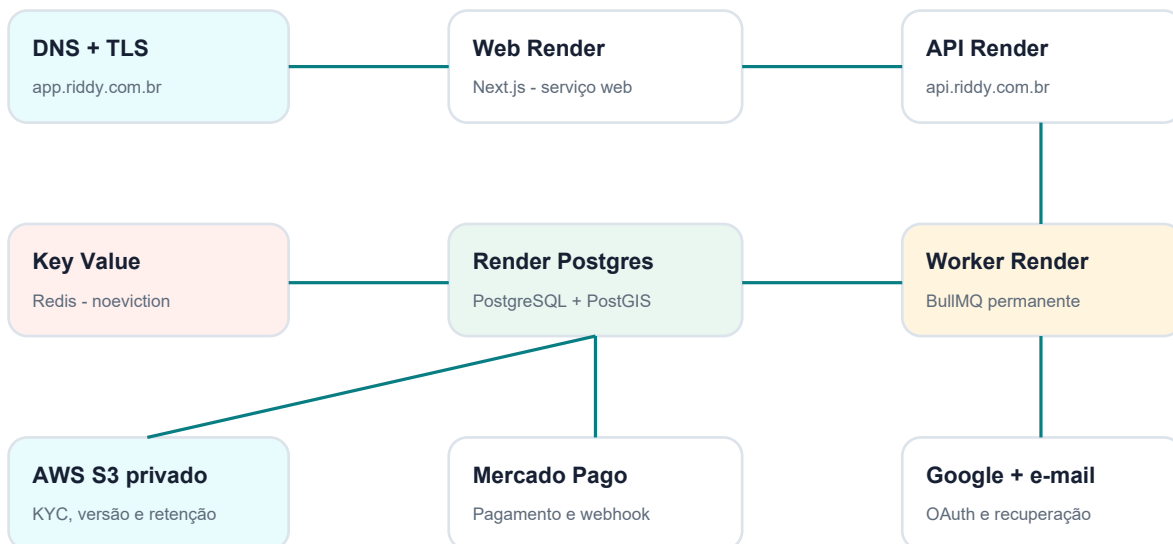
A Railway é a alternativa recomendada caso simplicidade e velocidade de implantação tenham prioridade sobre recursos operacionais mais gerenciados. Sua oferta atual inclui monorepos, bancos, Redis e buckets S3 compatíveis.

Plano gratuito não é produção

Use planos pagos que não suspendam serviços, com persistência, backups e recursos mínimos definidos. Custos e limites mudam; dimensione após teste de carga e confirme a tabela vigente de cada fornecedor.

Topologia alvo

Produção inicial com baixa complexidade e separação de riscos



Região única no início; rede privada para banco e Redis; somente Web, API e webhooks expostos.

CONFIGURAÇÃO DOS SERVIÇOS

Serviço	Build	Start / health
Web	<code>pnpm install --frozen-lockfile && pnpm --filter @riddy/web build</code>	<code>pnpm --filter @riddy/web start</code>
API	<code>pnpm install --frozen-lockfile && pnpm --filter @riddy/api build</code>	<code>pnpm --filter @riddy/api start / health/ready</code>
Worker	Mesmo build da API	<code>pnpm start:worker / sem URL pública</code>
Migrações	Artefato da mesma revisão	<code>pnpm db:migrate antes de promover</code>

A API deve usar `JOBS_WORKER_ENABLED=false`. O serviço worker executa o consumidor dedicado e compartilha `DATABASE_URL`, `REDIS_URL`, `REDIS_KEY_PREFIX` e credenciais de entrega de e-mail. O scheduler deve existir em uma única instância para evitar duplicação operacional.

Passo a passo de produção

Fases 0 a 3 - preparar, contratar, configurar e migrar

0

Definir ambientes e domínio

Criar staging e production separados. Reservar app.riddy.com.br e api.riddy.com.br. Definir responsáveis, orçamento, região e política de dados/KYC.

1

Provisionar dados

Criar Render Postgres pago na mesma região; habilitar PostGIS; criar Key Value com noeviction; criar bucket AWS S3 privado, versionamento, criptografia e lifecycle.

2

Criar os três serviços

Conectar o repositório no Render e criar Web Service para Next.js, Web Service para a API e Background Worker para BullMQ. Usar a branch main e Node 22.x.

3

Cadastrar segredos

Distribuir variáveis por serviço. Web recebe apenas URLs e flags públicas. API recebe todos os segredos necessários. Worker recebe somente banco, Redis, webhook de e-mail e configurações de jobs.

ORDEM DA MIGRAÇÃO

Ordem	Ação	Condição de sucesso
1	Backup/snapshot antes da mudança	Arquivo restaurável em destino separado.
2	Habilitar PostGIS	SELECT PostGIS_Version() responde.
3	Executar pnpm db:migrate	Todas as migrações registradas sem erro.
4	Não executar seed produtivo	Nenhuma conta demo ou dado fictício criado.
5	Subir API e worker	Readiness 200 e fila processando.
6	Subir Web	Home, login e catálogo usando a API produtiva.

Passo a passo de produção

Fases 4 a 7 - integrar, validar, publicar e operar

4

Configurar integrações

Google: projeto produtivo, domínio, redirect HTTPS e consent screen. Mercado Pago: credenciais TEST, webhook e assinatura; depois credenciais produtivas. E-mail: endpoint webhook e segredo.

5

Executar homologação

Validar cadastro, login Google, recuperação, KYC, reserva concorrente, cartão, Pix, webhook, cancelamento, reembolso, mensagens, permissões e suspensão administrativa.

6

Ligar entrega contínua

Manter GitHub Actions como gate. No Render, selecionar Auto-Deploy: After CI Checks Pass. Executar migrações com uma etapa controlada e smoke test após cada deploy.

7

Fazer go-live gradual

Começar com grupo limitado, monitorar erros e latências, manter rollback pronto e aumentar o tráfego somente após um ciclo completo de reserva e pagamento.

CHECKLIST DE INTEGRAÇÕES

Integração	Teste obrigatório antes do go-live
Google	Login novo e recorrente, cancelamento, state/PKCE, redirect e cookie Secure.
Mercado Pago	Cartão aprovado/recusado, Pix pendente/pago, repetição de webhook, refund e idempotência.
E-mail	Token não aparece em logs; link expira; retry BullMQ e entrega observável.
S3/KYC	Upload assinado, MIME real, checksum, leitura temporária, exclusão e acesso por papel.
Banco/Redis	Falha controlada, readiness, reconnect, backup e restauração.

Configuração por ambiente

Modelo de distribuição das variáveis - nunca coloque valores reais no repositório

Destino	Variáveis principais	Regra
Web	NEXT_PUBLIC_API_URL, NEXT_PUBLIC_SITE_URL, NEXT_PUBLIC_STORAGE_ORIGIN, NEXT_PUBLIC_GOOGLE_AUTH_ENABLED, API_URL	Somente valores públicos.
API	NODE_ENV, APP_VERSION, API_PORT, CORS_ORIGIN, WEB_URL, TRUST_PROXY, DATABASE_URL, REDIS_URL	Base produtiva obrigatória.
API/Auth	GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, GOOGLE_REDIRECT_URI, AUTH_RESET_WEBHOOK_URL, AUTH_RESET_WEBHOOK_SECRET	Segredos somente no servidor.
API/Pagamento	MERCADO_PAGO_PUBLIC_KEY, MERCADO_PAGO_ACCESS_TOKEN, MERCADO_PAGO_WEBHOOK_SECRET, MERCADO_PAGO_WEBHOOK_URL	Configurar o grupo completo.
API/Storage	S3_ENDPOINT, S3_REGION, S3_ACCESS_KEY, S3_SECRET_KEY, S3_KYC_BUCKET, S3_FORCE_PATH_STYLE	Bucket privado e HTTPS.
API/Operação	SWAGGER_ENABLED=false, AUTH_EXPOSE_RESET_TOKEN=false, LOG_LEVEL=info, JOBS_WORKER_ENABLED=false	Defaults seguros.
Worker	DATABASE_URL, REDIS_URL, REDIS_KEY_PREFIX, AUTH_RESET_WEBHOOK_URL, AUTH_RESET_WEBHOOK_SECRET	Menor conjunto de segredos.

Validação fail-fast existente

Em NODE_ENV=production, a API rejeita URLs públicas sem HTTPS, senhas locais conhecidas, reset token exposto e ausência de banco, Redis, storage, webhook de recuperação, proxy confiável e WEB_URL.

DNS E CALLBACKS DE EXEMPLO

Campo	Exemplo de produção
WEB_URL / CORS_ORIGIN	https://app.riddy.com.br
NEXT_PUBLIC_API_URL	https://api.riddy.com.br
GOOGLE_REDIRECT_URI	https://api.riddy.com.br/api/v1/auth/google/callback
MERCADO_PAGO_WEBHOOK_URL	https://api.riddy.com.br/api/v1/payments/webhook/mercado-pago

Os domínios acima são exemplos. Substituir pelo domínio realmente adquirido e manter os valores idênticos nos painéis Google, Mercado Pago, DNS, Web e API.

Gate de go-live

Crítérios que devem estar verdes antes de aceitar tráfego e dinheiro reais

Área	Crítério de aprovação
Segurança	Segredos no cofre; TLS; CORS; cookies Secure; Swagger off; dependências auditadas; revisão de permissões.
Dados	Migrações; PostGIS; backup automático; restauração ensaiada; retenção e exclusão de KYC aprovadas.
Pagamentos	Conta e credenciais produtivas; webhook assinado; conciliação; cancelamento/refund; política financeira.
Identidade	OAuth produtivo; domínio verificado; privacidade e termos publicados; KYC com acesso auditado.
Confiabilidade	Liveness/readiness; logs centralizados; alertas; limites; teste de carga; rollback testado.
Operação	Responsáveis, runbook, canal de incidente, suporte ao usuário e janela de deploy definidos.

COMANDOS DO GATE TÉCNICO

Comando	Finalidade
<code>pnpm install --frozen-lockfile</code>	Reproduzir exatamente o lockfile.
<code>pnpm format:check && pnpm lint</code>	Formatação e qualidade estática.
<code>pnpm typecheck && pnpm test</code>	Tipos e regressões automatizadas.
<code>pnpm db:check</code>	Integridade do histórico de migrações.
<code>pnpm audit --prod --audit-level=high</code>	Bloquear vulnerabilidades altas conhecidas.
<code>pnpm build</code>	Gerar artefatos de produção de Web e API.

Decisão final

O código está em condição de homologação. A produção só deve ser aprovada quando todos os itens desta página tiverem evidência, responsável e data. Não tratar credenciais preenchidas como sinônimo de prontidão operacional.

Primeiros 30 dias em produção

Operação gradual, evidências e melhoria contínua

Período	Ações
Dia 0	Snapshot, migração, deploy, smoke test, validação de callbacks e monitoramento intensivo.
Dias 1-3	Grupo limitado; revisar erros 5xx, pagamentos, filas, uploads, sessões e latência de banco.
Semana 1	Executar restauração de backup em ambiente isolado; ajustar alertas e limites.
Semana 2	Teste de carga baseado no tráfego observado; revisar consultas, cache e custos.
Semanas 3-4	Revisar incidentes, chargebacks, recusas KYC, suporte e roadmap; decidir expansão.

MÉTRICAS MÍNIMAS

Camada	Medir e alertar
Web/API	Disponibilidade, p95/p99, 4xx/5xx, request ID, deploy e consumo de CPU/memória.
Postgres	Conexões, consultas lentas, locks, disco, backup e replicação/restauração.
Redis/BullMQ	Memória, conexões, fila aguardando, jobs falhos, retries e idade do job mais antigo.
Negócio	Cadastro, OAuth, busca, reserva, aprovação de pagamento, Pix, reembolso e conversão.
KYC	Uploads falhos, tempo de análise, acessos a documentos, rejeições e exclusões.

Rollback

Manter a revisão anterior pronta para redeploy. Migrações devem ser compatíveis com rollback de aplicação; mudanças destrutivas exigem estratégia expand/contract e backup verificado antes da promoção.

Render - tipos de serviço, Postgres e Key Value

<https://render.com/docs/service-types>

Render - background workers e BullMQ

<https://render.com/docs/background-workers>

Render - extensões PostgreSQL/PostGIS

<https://render.com/docs/postgresql-extensions>

Render - deploy após CI e health checks

<https://render.com/docs/deloys>

Supabase - PostGIS

<https://supabase.com/docs/guides/database/extensions/postgis>

Supabase - compatibilidade S3 e limites

<https://supabase.com/docs/guides/storage/s3/compatibility>

Supabase - backups

<https://supabase.com/docs/guides/platform/backups>

Supabase - limites de Edge Functions

<https://supabase.com/docs/guides/functions/limits>

Railway - monorepos

<https://docs.railway.com/deployments/monorepo>

Railway - storage buckets S3

<https://docs.railway.com/storage-buckets>

Google - requisitos de verificação OAuth

<https://support.google.com/cloud/answer/13464321>

Google - ambientes de teste e produção

<https://support.google.com/cloud/answer/13464323>

Mercado Pago - Webhooks

<https://www.mercadopago.com.br/developers/pt/docs/your-integrations/notifications/webhooks>

Mercado Pago - credenciais

<https://www.mercadopago.com.br/developers/pt/docs/credentials>

AWS S3 - criptografia

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingEncryption.html>

AWS S3 - Object Lock e versionamento

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html>

Base interna

O inventário técnico e o estado das credenciais foram obtidos do README, .env.example, configurações, package.json, Docker Compose e documentos de arquitetura da Riddy. Nenhum valor secreto foi incluído neste PDF.

Este documento é um plano técnico, não substitui análise jurídica, contábil, de segurança ofensiva ou contrato de nível de serviço com fornecedores.